

DevOps avancé
Année universitaire 2026–2027

Travaux pratiques, TP1

Continuous Deployment avec GitOps et ArgoCD

Un dépôt Git décrit l'état désiré du cluster. ArgoCD le surveille, l'applique, corrige toute dérive et permet de revenir à une version stable.

Réalisé par

Kacem Mathlouthi
Oussema Kraiem
Med Amin Haouas

Dépôt

`github.com/KacemMathlouthi/tp-devops`

Image

`ghcr.io/kacemmathlouthi/tp-devops`

Table des matières

Présentation générale 3
Objectif 3
Application support 3
Construction de l'image 3
Stack technique 4
Architecture 4
Étape 1 : structure du dépôt Git 5
Organisation 5
Base Kustomize 5
Overlay `dev` 6
Étape 2 : installation et configuration d'ArgoCD 7
Cluster Kubernetes 7
Installation d'ArgoCD 7
Accès à l'interface 8
Création de l'Application 9
Démonstration 1 : déploiement continu automatisé 12
Préparation : construction de la version v2 12
Modification de l'état désiré dans Git 12
Synchronisation automatique par ArgoCD 12
Démonstration 2 : détection du drift et self-healing 15
Suppression du Deployment 15
Réconciliation automatique 15
Drift sur un champ : nombre de réplicas 16
Démonstration 3 : Sync Status et Health Status 19
Rappel : deux indicateurs indépendants 19
Introduction d'une erreur dans Git 19
Synced mais Progressing, puis Degraded 19
Démonstration 4 : historique et rollback 22
Désactivation de l'auto-sync 22
Historique des déploiements 22
Rollback 23
Retour à Git comme source de vérité 24
Conclusion 27

Présentation générale

Objectif

Ce TP met en place un pipeline GitOps de bout en bout : l'état désiré de l'application et de ses ressources Kubernetes est décrit de façon déclarative dans un dépôt Git, et ArgoCD, installé dans le cluster, se charge de le faire converger vers cet état. Aucun `kubectl apply` n'est lancé depuis l'extérieur pour déployer : le cluster tire (pull) sa configuration depuis Git.

Le TP doit démontrer cinq concepts :

Single Source of Truth	Tout l'état désiré vit dans Git : manifests Kustomize et définition de l'Application ArgoCD.
Pull-based CD	ArgoCD surveille le dépôt et applique automatiquement chaque commit.
Drift et self-healing	Toute modification manuelle du cluster est détectée puis annulée.
Sync et Health	Un manifeste conforme à Git (Synced) peut produire des pods en échec (Degraded).
Rollback et historique	Retour instantané à une révision stable depuis l'onglet History and Rollback.

Application support

Le sujet laissant le choix de l'application, nous avons retenu une petite API `FastAPI` (Python 3.12, dépendances gérées par `uv`) qui expose trois routes :

- `GET /` : page HTML affichant la version déployée et le nom du pod qui a servi la requête ;
- `GET /version` : la même information au format JSON ;
- `GET /healthz` : sonde utilisée par Kubernetes (readiness et liveness).

La version affichée est injectée au build de l'image (`ARG APP_VERSION`) : chaque démonstration est ainsi visible directement dans le navigateur, et le nom du pod permet de constater la répartition de charge entre réplicas.

Construction de l'image

Le dépôt contient le code applicatif (`app/`) et une GitHub Action (`.github/workflows/build.yml`) qui, à chaque tag Git `v*`, construit l'image et la publie sur GitHub Container Registry :

```
git tag v2 && git push origin v2
# → ghcr.io/kacemmathlouthi/tp-devops:v2
```

Séparation CI / CD

La CI (GitHub Actions) se contente de produire une image versionnée ; elle n'a aucun accès au cluster. Le déploiement (CD) est entièrement piloté par ArgoCD, qui tire l'état désiré depuis Git : changer la version déployée revient à modifier le tag dans l'overlay Kustomize et à pousser le commit.

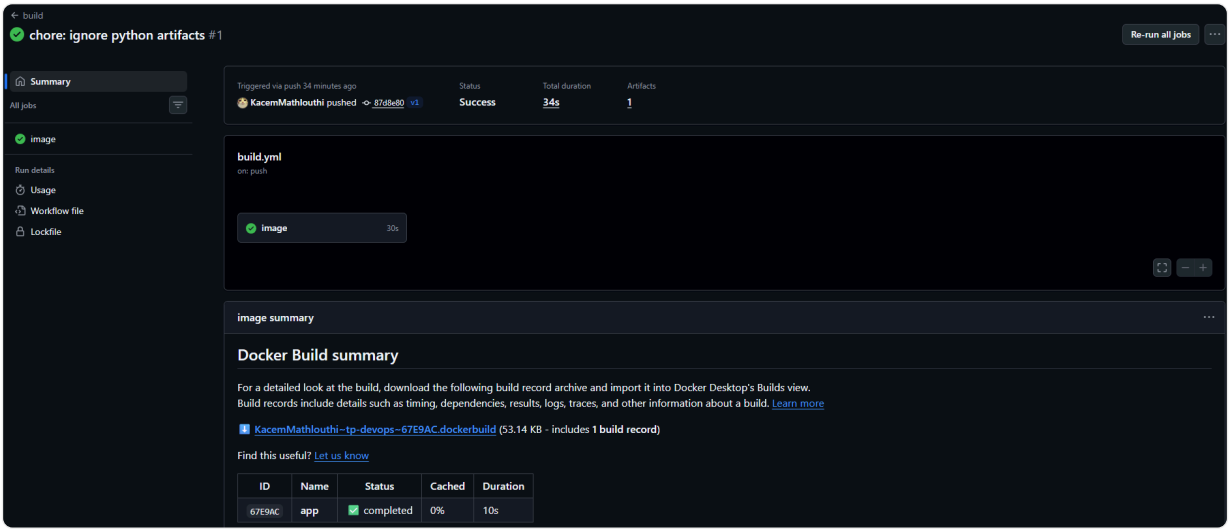
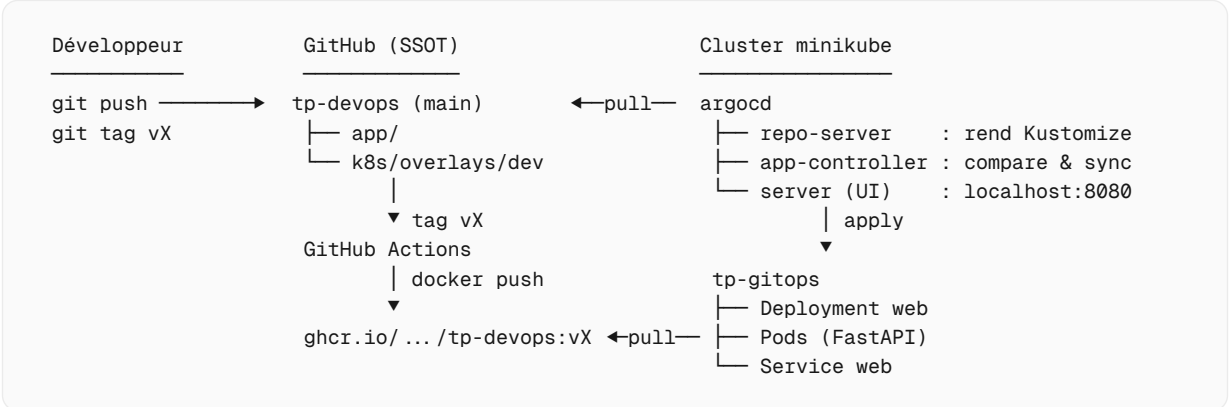


Fig. 1. GitHub Actions : le push du tag `v1` déclenche le workflow `build` (34 s) qui construit et publie l'image

Stack technique

Cluster	minikube (driver Docker) sous WSL 2
GitOps	ArgoCD (manifests officiels, namespace `argocd`)
Manifests	Kustomize : une `base` et un overlay `dev`
Dépôt	GitHub public `KacemMathlouthi/tp-devops`, branche `main`
Application	FastAPI (Python 3.12, uv), image Docker non-root
CI	GitHub Actions vers `ghcr.io/kacemmathlouthi/tp-devops:`

Architecture



Étape 1 : structure du dépôt Git

Organisation

Un seul dépôt contient l'ensemble du TP : code applicatif, pipeline de build et manifests. ArgoCD ne lit que le chemin `k8s/overlays/dev` ; le reste du dépôt (code, rapport, définition de l'Application) ne perturbe donc pas la synchronisation.

```
tp-devops/
├── app/                          # code applicatif
│   ├── main.py                   # API FastAPI
│   ├── pyproject.toml, uv.lock   # dépendances (uv)
│   └── Dockerfile
├── .github/workflows/build.yml   # build + push de l'image sur tag vX
├── k8s/
│   ├── base/                     # ressources communes
│   │   ├── kustomization.yaml
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── overlays/
│       └── dev/
│           └── kustomization.yaml  # namespace, replicas, tag d'image
├── argocd/
│   └── application.yaml          # l'Application ArgoCD (déclarative elle aussi)
└── compte-rendu/                 # ce rapport
```

Base Kustomize

La base décrit l'application indépendamment de tout environnement : un `Deployment` et un `Service`, auxquels Kustomize ajoute des labels communs.

```
# k8s/base/kustomization.yaml
labels:
  - pairs:
      app.kubernetes.io/name: web
      app.kubernetes.io/part-of: tp-gitops
resources:
  - deployment.yaml
  - service.yaml
```

Points notables du `Deployment` :

- sondes `readinessProbe` et `livenessProbe` sur `/healthz` : c'est sur elles que repose le Health Status calculé par ArgoCD ;
- variable `POD_NAME` injectée via l'API downward, affichée par l'application ;
- requests/limits CPU et mémoire ;
- `progressDeadlineSeconds: 60` : en cas d'image introuvable, Kubernetes marque le rollout en échec au bout d'une minute (au lieu de 10 par défaut) et ArgoCD affiche alors Degraded.

Le `Service` (`ClusterIP`, port 80 → 8000) répartit le trafic entre les pods portant le label `app: web`.

Overlay `dev`

L'overlay est le seul fichier modifié pendant les démonstrations : il fixe le namespace, le nombre de réplicas et le tag de l'image.

```
# k8s/overlays/dev/kustomization.yaml
namespace: tp-gitops
resources:
  - ../../base
replicas:
  - name: web
    count: 2
images:
  - name: ghcr.io/kacemmathlouthi/tp-devops
    newTag: v1
```

Le rendu final peut être vérifié localement sans cluster :

```
kubectl kustomize k8s/overlays/dev
```

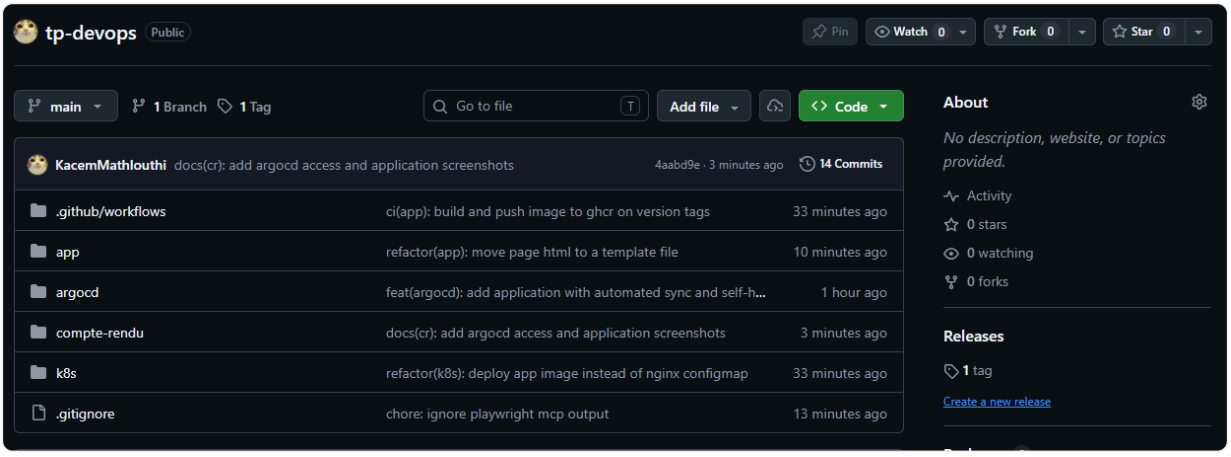


Fig. 2. Dépôt public `KacemMathlouthi/tp-devops` : code applicatif, workflow CI, manifests Kustomize, Application ArgoCD et rapport ; le package GHCR `tp-devops` est lié au dépôt

Étape 2 : installation et configuration d'ArgoCD

Cluster Kubernetes

Le cluster est un minikube mono-nœud utilisant le driver Docker, sous WSL 2. Deux vCPU et environ 4 Go de RAM suffisent pour Kubernetes, ArgoCD (7 pods) et l'application.

```
minikube start --driver=docker --cpus=2 --memory=4g
kubectl config use-context minikube
kubectl get nodes
```

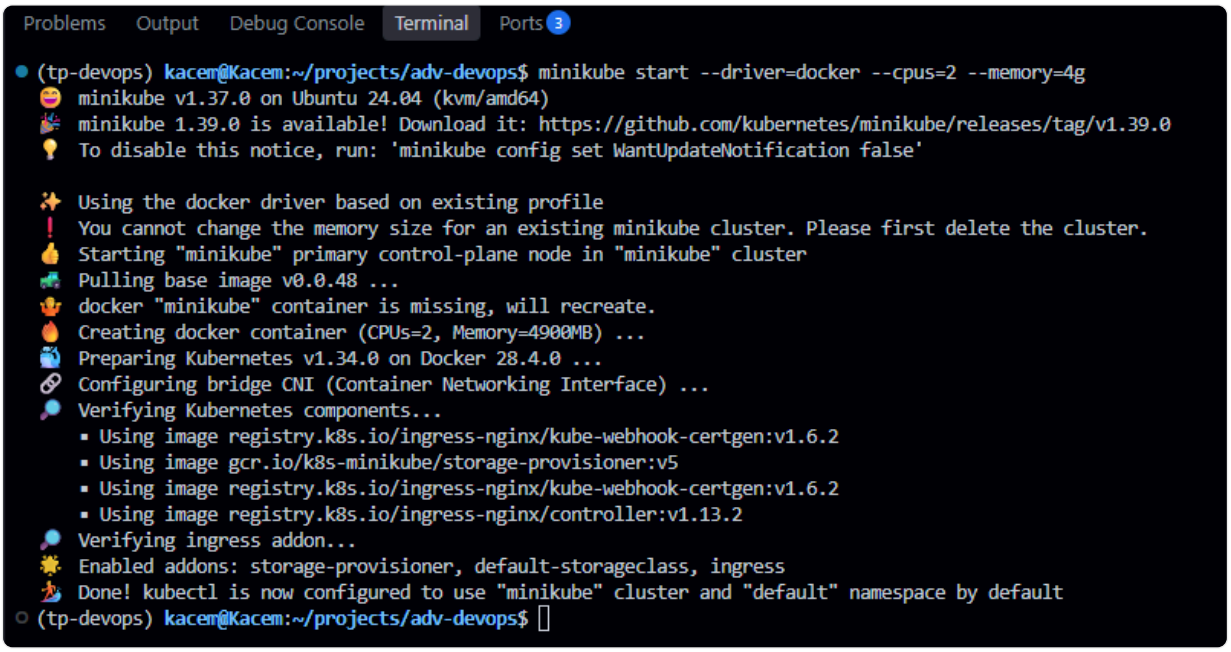


Fig. 3. Démarrage du cluster minikube (Kubernetes v1.34.0, driver Docker) ; `kubectl` est configuré sur le contexte `minikube`

Installation d'ArgoCD

ArgoCD est installé à partir des manifests officiels dans son propre namespace. L'option `--server-side` est nécessaire car certaines CRD dépassent la taille maximale de l'annotation `last-applied-configuration`.

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
```

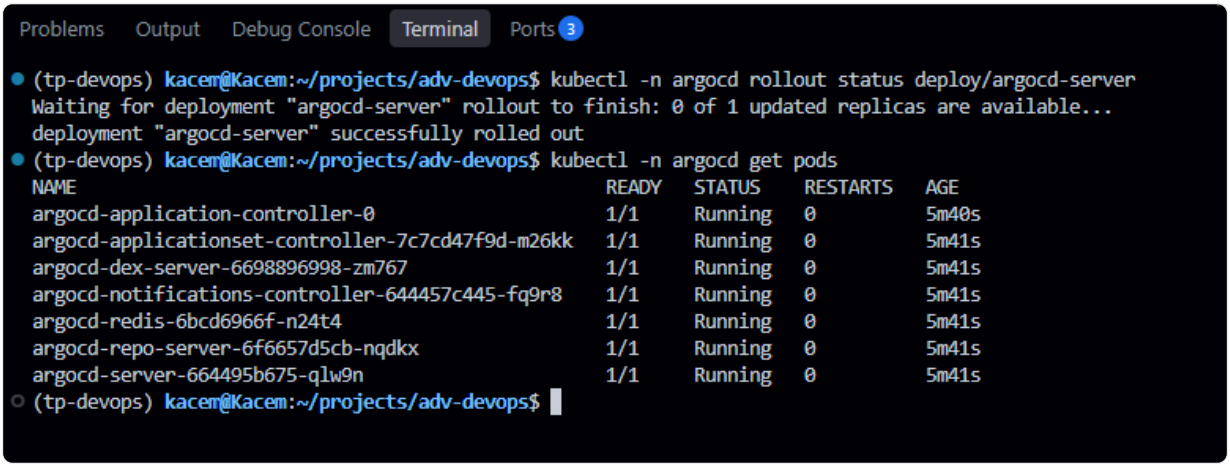


Fig. 4. Fin du rollout de `argocd-server` puis les 7 pods ArgoCD en état `Running` : application-controller, applicationset-controller, dex, notifications, redis, repo-server et server

Accès à l'interface

```
# mot de passe initial de l'utilisateur admin
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
# exposition locale de l'UI
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

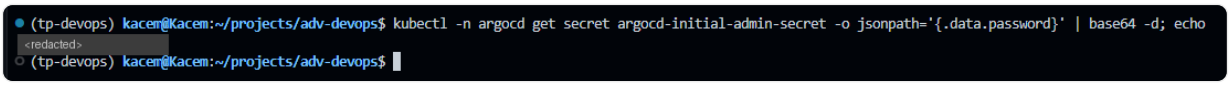


Fig. 5. Récupération du mot de passe initial de l'utilisateur `admin` (valeur masquée)

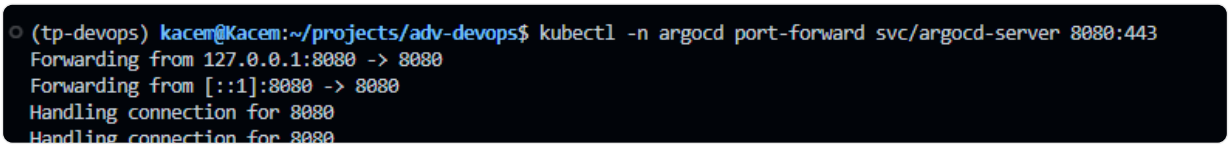


Fig. 6. Port-forward du service `argocd-server` vers `localhost:8080`

L'interface est alors accessible sur `https://localhost:8080` (certificat auto-signé). Après connexion, ArgoCD (v3.5.5) ne gère encore aucune application.

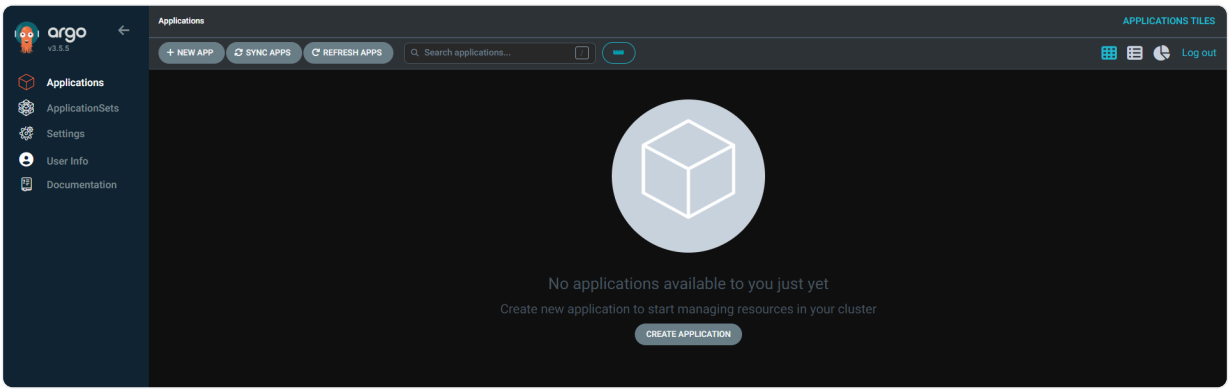


Fig. 7. Tableau de bord ArgoCD après la première connexion : aucune application

Création de l'Application

L'Application est elle-même décrite dans Git (`argocd/application.yaml`) : la configuration de l'outil GitOps suit le même principe déclaratif que l'application.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tp-gitops
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/KacemMathlouthi/tp-devops.git
    targetRevision: main
    path: k8s/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: tp-gitops
  syncPolicy:
    automated:
      prune: true      # supprime ce qui a disparu de Git
      selfHeal: true   # annule les modifications manuelles
    syncOptions:
      - CreateNamespace=true
```

```
kubectl apply -f argocd/application.yaml
kubectl -n tp-gitops get pods -w
```

C'est la seule commande `kubectl apply` du TP : à partir de là, ArgoCD crée lui-même le namespace `tp-gitops` et les ressources décrites dans Git. Les deux pods passent de `ContainerCreating` à `Running` puis `1/1` (readiness probe validée) en moins de 30 secondes.

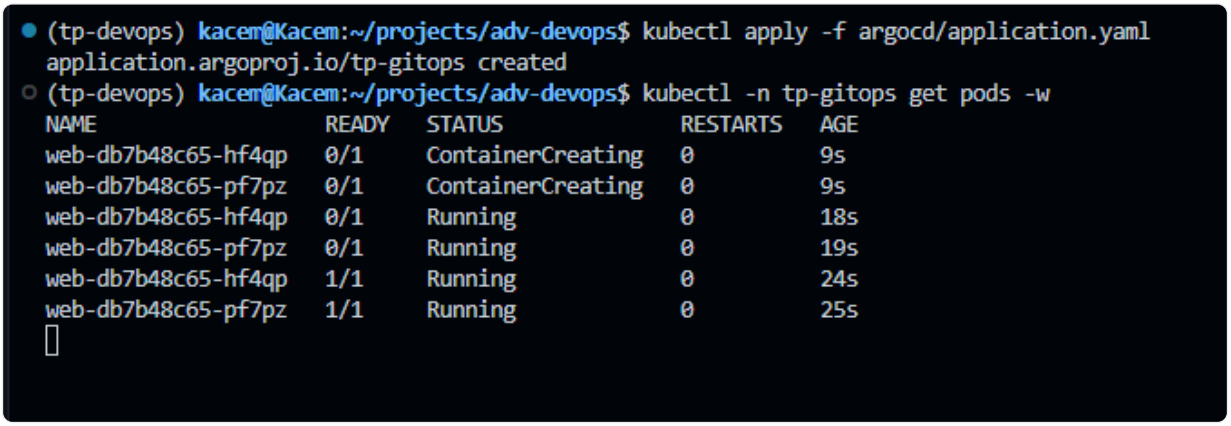


Fig. 8. Création de l'Application puis démarrage des deux pods `web` déployés par ArgoCD

Options de synchronisation

- **automated** : ArgoCD synchronise dès qu'un nouveau commit est détecté, sans clic sur Sync.
- **selfHeal** : si l'état du cluster s'écarte de Git (drift), ArgoCD réapplique les manifests.
- **prune** : une ressource retirée de Git est supprimée du cluster.

– ArgoCD interroge Git toutes les 3 minutes par défaut ; le bouton Refresh force une vérification immédiate.

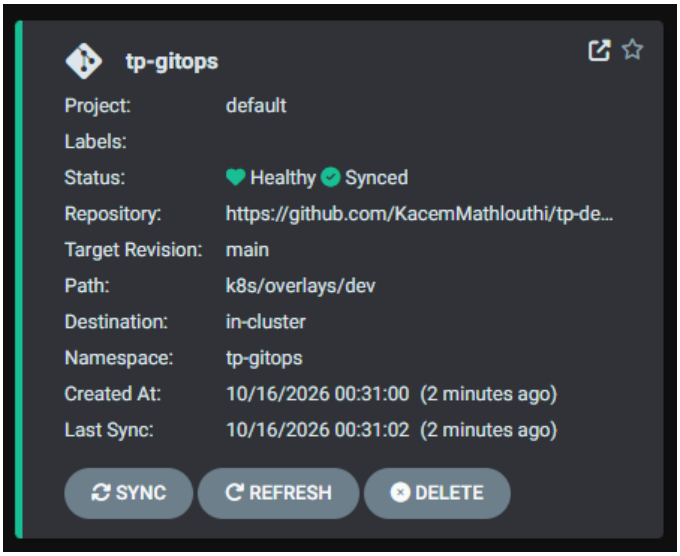


Fig. 9. Tuile de l'application `tp-gitops` : dépôt, révision `main`, chemin `k8s/overlays/dev`, statut Healthy et Synced

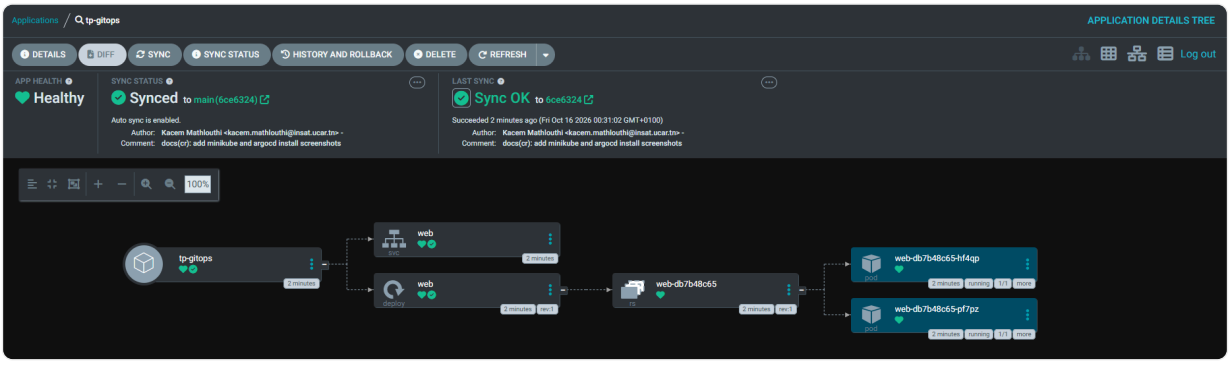


Fig. 10. Arbre des ressources : Service et Deployment `web`, ReplicaSet et deux pods. Synchronisé automatiquement sur le commit `6ce6324` (Auto sync is enabled)

L'en-tête de la vue détaillée illustre déjà la distinction au cœur du TP : App Health (état réel des ressources, ici Healthy) et Sync Status (conformité avec Git, ici Synced sur `main`) sont deux indicateurs indépendants.

```
kubectl -n tp-gitops port-forward svc/web 8081:80   # http://localhost:8081
```



Fig. 11. Application servie par le cluster : version `v1`, réponse du pod `web-db7b48c65-hf4qp`

Démonstration 1 : déploiement continu automatisé

Objectif : montrer qu'un simple commit suffit à modifier le cluster, sans aucune commande `kubectl`.

Préparation : construction de la version v2

La nouvelle version de l'application (refonte de la page) est publiée en poussant un tag Git. La CI produit l'image `ghcr.io/kacemmathlouthi/tp-devops:v2` mais ne touche pas au cluster : tant que Git n'y fait pas référence, rien n'est déployé.

```
git tag v2 && git push origin v2
```

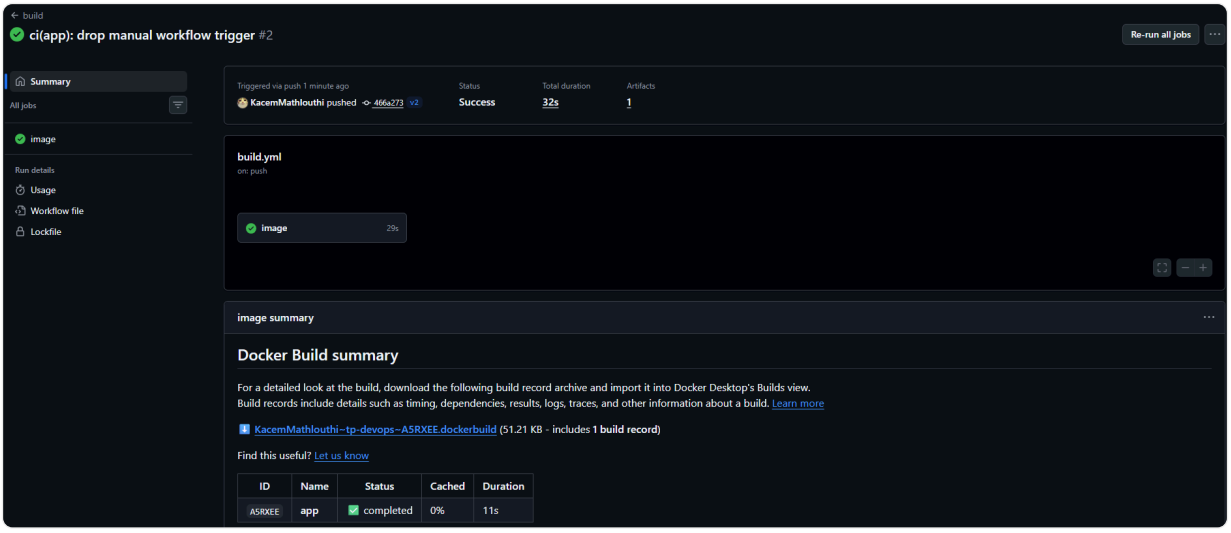


Fig. 12. GitHub Actions : le tag `v2` déclenche le build et la publication de l'image `tp-devops:v2` (32 s)

Modification de l'état désiré dans Git

Le déploiement de la nouvelle version se résume à changer une ligne dans l'overlay, puis à pousser le commit. Aucune commande n'est envoyée au cluster.

```
# k8s/overlays/dev/kustomization.yaml
 images:
   - name: ghcr.io/kacemmathlouthi/tp-devops
-    newTag: v1
+    newTag: v2
```

```
git commit -am "feat(k8s): deploy v2" && git push
```

Synchronisation automatique par ArgoCD

ArgoCD interroge le dépôt périodiquement (toutes les 3 minutes par défaut) ; le bouton Refresh force une vérification immédiate sans déclencher de synchronisation manuelle. Dès que le nouveau commit

est détecté, la politique `automated` applique le Deployment modifié et Kubernetes effectue un rolling update.

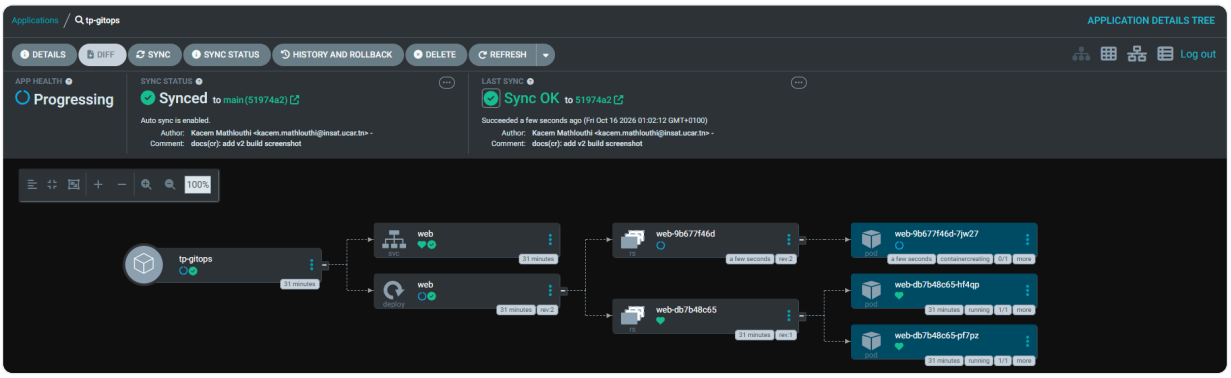


Fig. 13. Pendant la synchronisation : l'application est Synced sur le nouveau commit mais Progressing. Un nouveau ReplicaSet `web-9b677f46d` (`rev:2`, image v2) démarre son premier pod (`ContainerCreating`) tandis que les deux pods v1 continuent de servir le trafic

Le rolling update respecte les valeurs par défaut du Deployment (`maxSurge: 25%`, `maxUnavailable: 25%`) : un pod v2 est créé, et ce n'est qu'une fois sa readiness probe validée qu'un pod v1 est supprimé. La capacité ne descend jamais sous deux pods prêts.

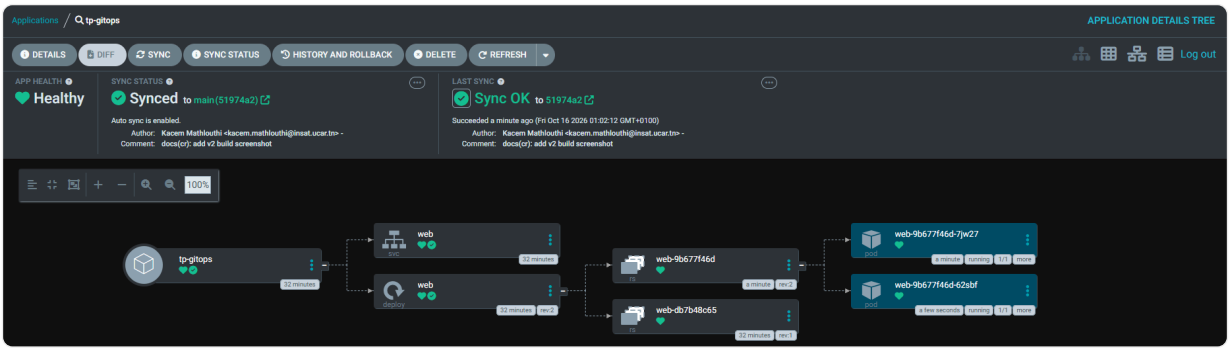


Fig. 14. Fin du rollout : Healthy et Synced. Les deux pods appartiennent au ReplicaSet v2 ; l'ancien ReplicaSet `web-db7b48c65` (`rev:1`) est conservé à 0 réplica comme historique Kubernetes (`revisionHistoryLimit`)

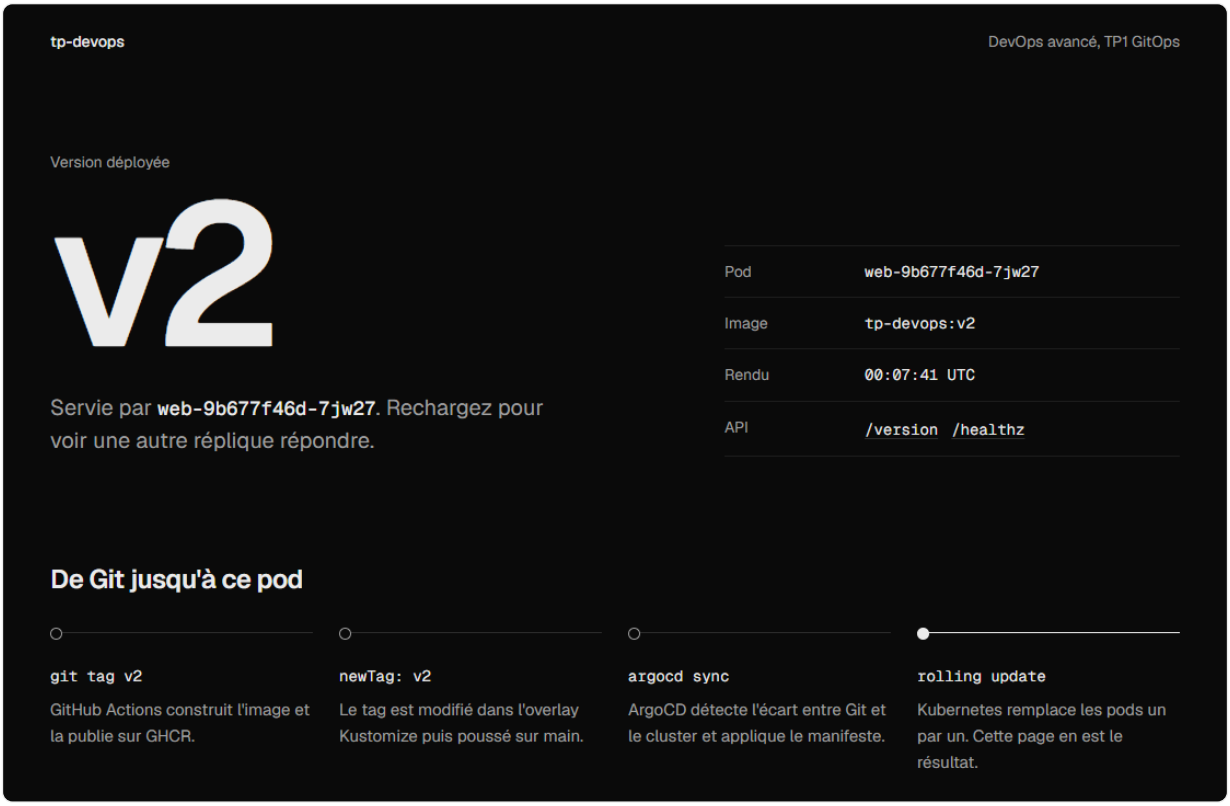


Fig. 15. Résultat visible côté utilisateur : la page affiche désormais la version `v2` (nouvelle interface), servie par le pod `web-9b677f46d-7jw27` du nouveau ReplicaSet

Résultat

Un simple commit dans Git a suffi à déployer la version v2 : ArgoCD a détecté l'écart entre l'état désiré et l'état réel, puis a appliqué le changement sans aucune intervention sur le cluster. Le statut de santé est passé de Progressing à Healthy à la fin du rolling update, sans interruption de service.

Démonstration 2 : détection du drift et self-healing

Objectif : altérer manuellement le cluster et montrer qu'ArgoCD rétablit l'état décrit dans Git.

Suppression du Deployment

On simule une erreur humaine (ou une action non autorisée) en supprimant directement le Deployment depuis le terminal, sans passer par Git :

```
kubectl -n tp-gitops delete deployment web
```

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops delete deployment web
deployment.apps "web" deleted from tp-gitops namespace
(tp-devops) kacem@Kacem:~/projects/adv-devops$
```

Fig. 16. Suppression manuelle du Deployment `web` : les deux pods v2 sont supprimés en cascade

L'état réel du cluster ne correspond plus à Git : la ressource `Deployment/web` déclarée dans l'overlay est absente. L'application-controller d'ArgoCD surveille en permanence les ressources qu'il gère (via des watches sur l'API Kubernetes) ; il détecte immédiatement l'écart et, comme `selfHeal: true` est activé, relance une synchronisation sans attendre le prochain polling de Git.

Réconciliation automatique

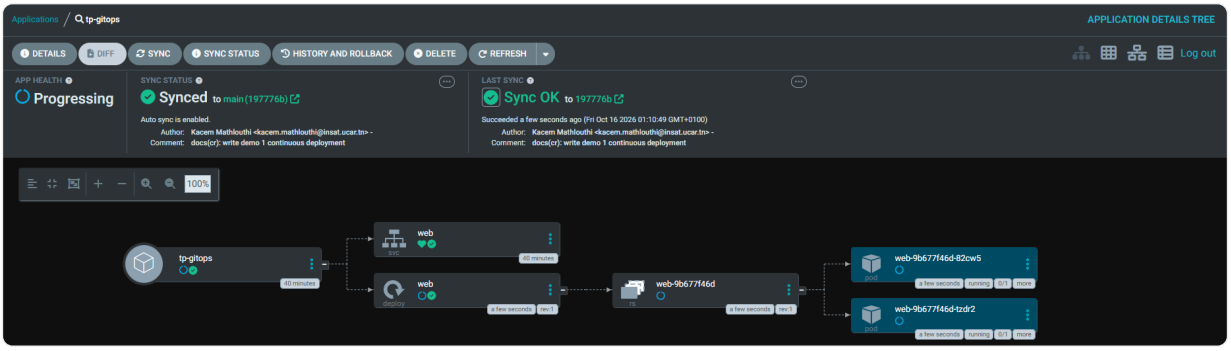


Fig. 17. Quelques secondes après la suppression : ArgoCD a déjà recréé le Deployment (`a few seconds`, `rev:1`) et son ReplicaSet ; les nouveaux pods démarrent (`0/1`), l'application est Progressing

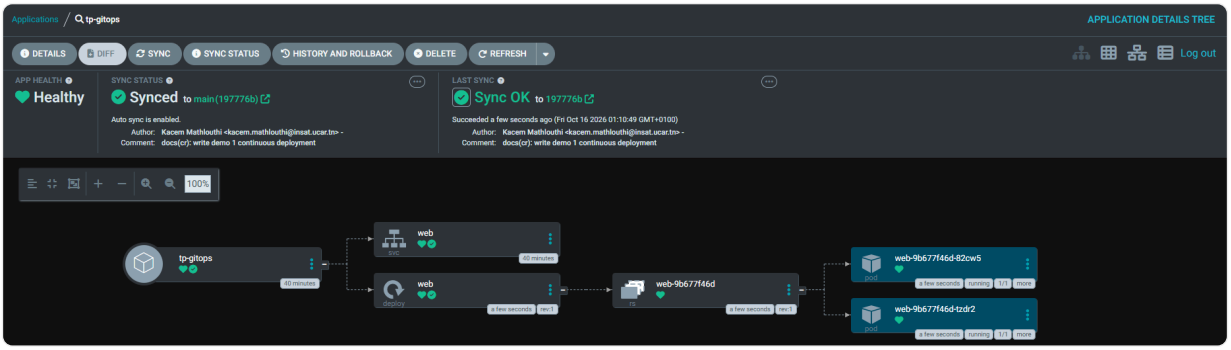


Fig. 18. Environ 15 secondes après la suppression : application de nouveau Healthy et Synced. Le Service, qui n'a pas été touché, garde son âge (40 minutes) ; seuls le Deployment, le ReplicaSet et les pods sont neufs

Deux détails confirment que la ressource a bien été recréée et non simplement conservée :

- le Deployment repart en révision `rev:1` : son historique Kubernetes (dont l'ancien ReplicaSet v1) a disparu avec lui ;
- les pods ont de nouveaux noms (`web-9b677f46d-82cw5`, `web-9b677f46d-tzdr2`) mais le même hash de template, car ils sont reconstruits à partir du même manifeste Git (image v2).

Drift sur un champ : nombre de réplicas

Le drift n'est pas forcément une suppression : une simple modification d'un champ est aussi détectée. On augmente manuellement le nombre de réplicas alors que Git en déclare deux :

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops scale deploy web --replicas=5
deployment.apps/web scaled
(tp-devops) kacem@Kacem:~/projects/adv-devops$
```

Fig. 19. Passage manuel du Deployment à 5 réplicas avec `kubectl scale`



Fig. 20. Quelques secondes plus tard : ArgoCD a rétabli `replicas: 2`. Les pods créés par le scale manuel (`btccd`, `xr9mz`) sont déjà en `terminating` (`0/1`) ; seuls les deux pods conformes à Git restent `running`

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops get pods -w
NAME                   READY   STATUS              RESTARTS   AGE
web-9b677f46d-82cw5    1/1     Running             0          8m44s
web-9b677f46d-tzdr2    1/1     Running             0          8m44s
web-9b677f46d-ckrfd    0/1     Pending             0          0s
web-9b677f46d-jprv5    0/1     Pending             0          0s
web-9b677f46d-zrkrf    0/1     Pending             0          0s
web-9b677f46d-ckrfd    0/1     Pending             0          0s
web-9b677f46d-jprv5    0/1     Pending             0          0s
web-9b677f46d-zrkrf    0/1     Pending             0          0s
web-9b677f46d-ckrfd    0/1     ContainerCreating   0          0s
web-9b677f46d-zrkrf    0/1     ContainerCreating   0          0s
web-9b677f46d-jprv5    0/1     ContainerCreating   0          0s
web-9b677f46d-ckrfd    0/1     Terminating         0          0s
web-9b677f46d-zrkrf    0/1     Terminating         0          0s
web-9b677f46d-jprv5    0/1     Terminating         0          0s
web-9b677f46d-zrkrf    0/1     Terminating         0          2s
web-9b677f46d-jprv5    0/1     Terminating         0          2s
web-9b677f46d-ckrfd    0/1     Terminating         0          2s
web-9b677f46d-zrkrf    1/1     Terminating         0          7s
web-9b677f46d-jprv5    1/1     Terminating         0          7s
web-9b677f46d-ckrfd    1/1     Terminating         0          7s
web-9b677f46d-zrkrf    0/1     Error               0          35s
web-9b677f46d-jprv5    0/1     Error               0          35s
web-9b677f46d-ckrfd    0/1     Error               0          35s
web-9b677f46d-zrkrf    0/1     Error               0          36s
web-9b677f46d-zrkrf    0/1     Error               0          36s
web-9b677f46d-jprv5    0/1     Error               0          36s
web-9b677f46d-jprv5    0/1     Error               0          36s
web-9b677f46d-jprv5    0/1     Error               0          36s
web-9b677f46d-ckrfd    0/1     Error               0          36s
web-9b677f46d-ckrfd    0/1     Error               0          36s
web-9b677f46d-ckrfd    0/1     Error               0          36s
```

Fig. 21. `kubectl get pods -w` pendant le scale : les trois pods supplémentaires passent en `Terminating` dès leur création (âge `0s`), les deux pods d'origine ne sont jamais touchés

Attention

Les trois pods supprimés finissent en `Error` au bout d'environ 35 s au lieu de s'arrêter proprement. Le signal `SIGTERM` envoyé par Kubernetes est arrivé pendant le démarrage d'uvicorn, avant l'installation de son gestionnaire de signaux. Or uvicorn tourne en PID 1 dans le conteneur, et le noyau ignore les signaux non gérés envoyés au PID 1 : le conteneur est donc tué par `SIGKILL` à l'expiration du délai de grâce (30 s, code 137). Un pod démarré depuis plus d'une seconde s'arrête en moins d'une seconde. La correction habituelle consiste à lancer l'application derrière un mini-init (`tini`).

La correction est si rapide que l'application ne passe même pas visiblement par l'état OutOfSync : Last Sync indique simplement une nouvelle synchronisation automatique réussie quelques secondes après la modification.

Résultat

Toute modification faite hors de Git est considérée comme un drift et annulée : le Deployment supprimé a été recréé en quelques secondes, sans intervention humaine. Git reste l'unique source de vérité ; pour modifier durablement le cluster, il faut passer par un commit.

Démonstration 3 : Sync Status et Health Status

Objectif : montrer qu'une application peut être parfaitement synchronisée avec Git tout en étant défaillante.

Rappel : deux indicateurs indépendants

Sync Status	compare l'état déclaré dans Git à l'état déclaré dans le cluster (les manifests). Valeurs : Synced, OutOfSync.
Health Status	évalue l'état opérationnel des ressources (pods prêts, rollout terminé…). Valeurs : Healthy, Progressing, Degraded, Missing…

Un manifeste peut donc être appliqué à la lettre tout en décrivant quelque chose qui ne fonctionne pas.

Introduction d'une erreur dans Git

On référence une image qui n'existe pas dans le registre :

```
# k8s/overlays/dev/kustomization.yaml
 images:
   - name: ghcr.io/kacemmathlouthi/tp-devops
-    newTag: v2
+    newTag: doesnotexist
```

```
git commit -am "feat(k8s): deploy nonexistent image" && git push
```

Synced mais Progressing, puis Degraded

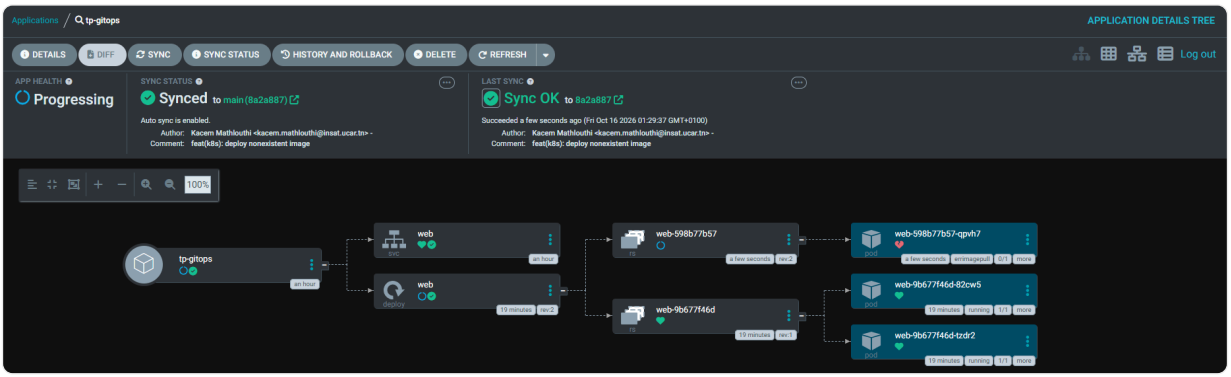


Fig. 22. Quelques secondes après la synchronisation : Synced sur le commit `8a2a887` mais Progressing. Le nouveau ReplicaSet `web-598b77b57` (`rev:2`) tente de démarrer un pod qui échoue en `errimagepull` ; les deux pods v2 restent `running`

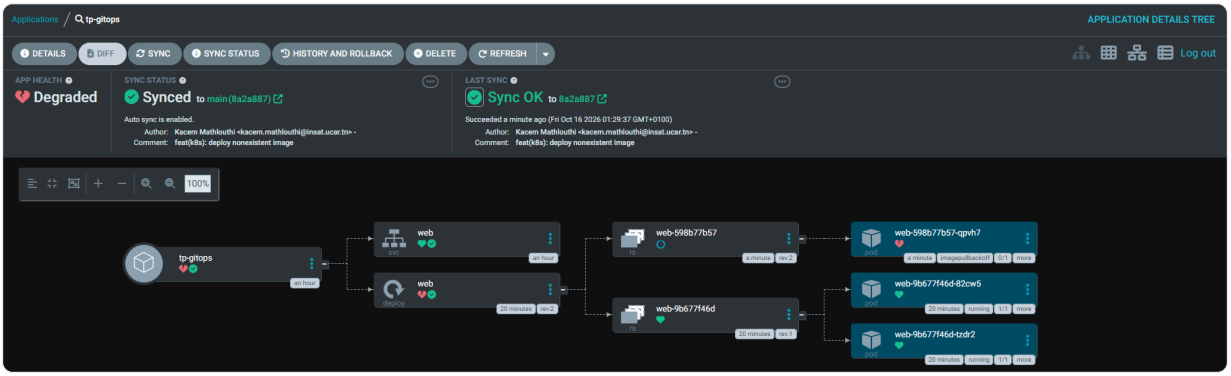


Fig. 23. Après l'expiration de `progressDeadlineSeconds` (60 s) : l'application est toujours Synced mais passe en Degraded. Le pod en échec est en `imagepullbackoff`, le Deployment porte un cœur rouge

Le passage de Progressing à Degraded vient de Kubernetes, pas d'ArgoCD : au bout de 60 s sans progression, le contrôleur de Deployment pose la condition `Progressing=False` (raison `ProgressDeadlineExceeded`), qu'ArgoCD traduit en Degraded.

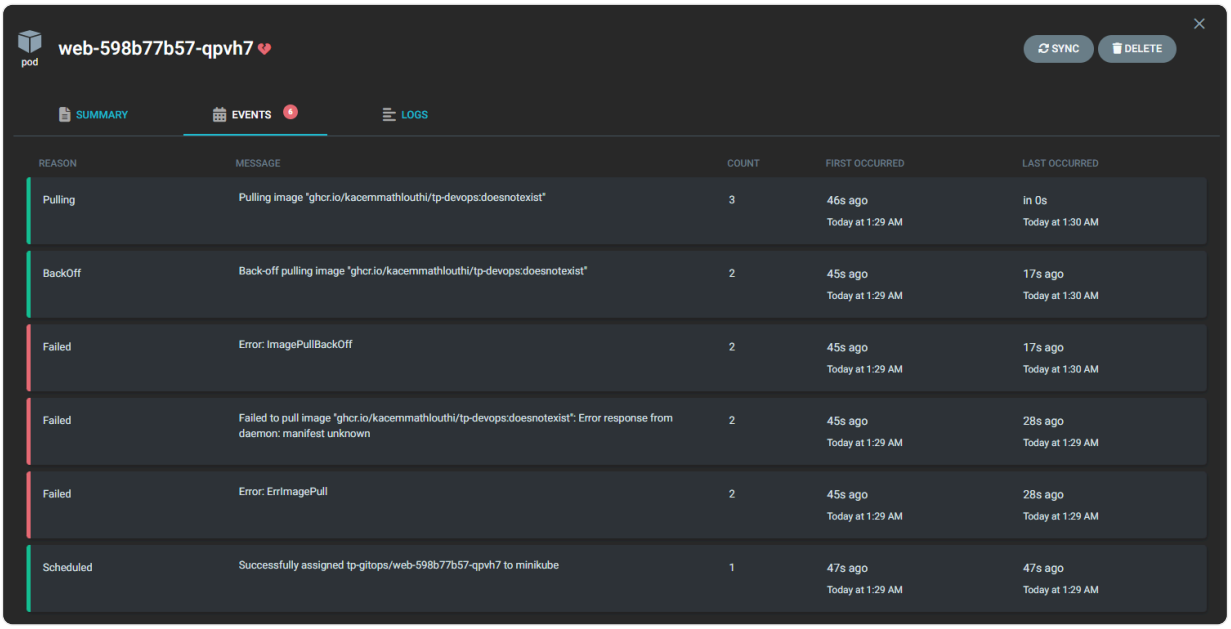


Fig. 24. Événements du pod en échec dans ArgoCD : `Failed to pull image « ghcr.io/kacemmathlouthi/tp-devops:doesnotexist »: manifest unknown`, puis `ErrImagePull` et `ImagePullBackOff` (Kubernetes réessaie avec un délai croissant)

```
(tp-devops) kacem@Kacem:~/projects/adv-devops$ kubectl -n tp-gitops get pods -w
NAME                   READY   STATUS              RESTARTS   AGE
web-9b677f46d-82cw5    1/1     Running             0          18m
web-9b677f46d-tzdr2    1/1     Running             0          18m
web-598b77b57-qpvh7    0/1     Pending             0          0s
web-598b77b57-qpvh7    0/1     Pending             0          0s
web-598b77b57-qpvh7    0/1     ContainerCreating   0          0s
web-598b77b57-qpvh7    0/1     ErrImagePull        0          2s
web-598b77b57-qpvh7    0/1     ImagePullBackOff    0          18s
web-598b77b57-qpvh7    0/1     ErrImagePull        0          30s
web-598b77b57-qpvh7    0/1     ImagePullBackOff    0          47s
web-598b77b57-qpvh7    0/1     ErrImagePull        0          62s
web-598b77b57-qpvh7    0/1     ImagePullBackOff    0          77s
web-598b77b57-qpvh7    0/1     ErrImagePull        0          2m2s
web-598b77b57-qpvh7    0/1     ImagePullBackOff    0          2m18s
```

Fig. 25. `kubectl get pods -w` : le nouveau pod alterne entre `ErrImagePull` et `ImagePullBackOff` avec des délais croissants (2 s, 18 s, 47 s, 77 s…), tandis que les deux pods v2 restent `Running 1/1`

Pas d'interruption de service

Le rolling update ne supprime un ancien pod qu'une fois le nouveau prêt. Le pod `doesnotexist` ne devenant jamais prêt, les deux pods v2 de l'ancien ReplicaSet continuent de servir le trafic : le déploiement est bloqué, mais l'application reste disponible.

Résultat

L'application est Synced : ArgoCD a correctement appliqué ce que Git décrit. Elle est pourtant Degraded, car ce que Git décrit est faux. La synchronisation garantit la conformité à la source de vérité, pas le bon fonctionnement ; c'est le Health Status qui révèle l'erreur applicative. Pour corriger, il faut revenir à un état stable, ce qui fait l'objet de la démonstration suivante.

Démonstration 4 : historique et rollback

Objectif : revenir instantanément à une version stable depuis l'onglet History and Rollback.

On part de l'état laissé par la démonstration 3 : l'application est Synced mais Degraded sur le commit `8a2a887` (image `doesnotexist`).

Désactivation de l'auto-sync

ArgoCD refuse un rollback tant que la synchronisation automatique est active : sans cela, la politique `automated` réappliquerait immédiatement la révision de Git, c'est-à-dire la version cassée. On la désactive depuis Details → Sync Policy.

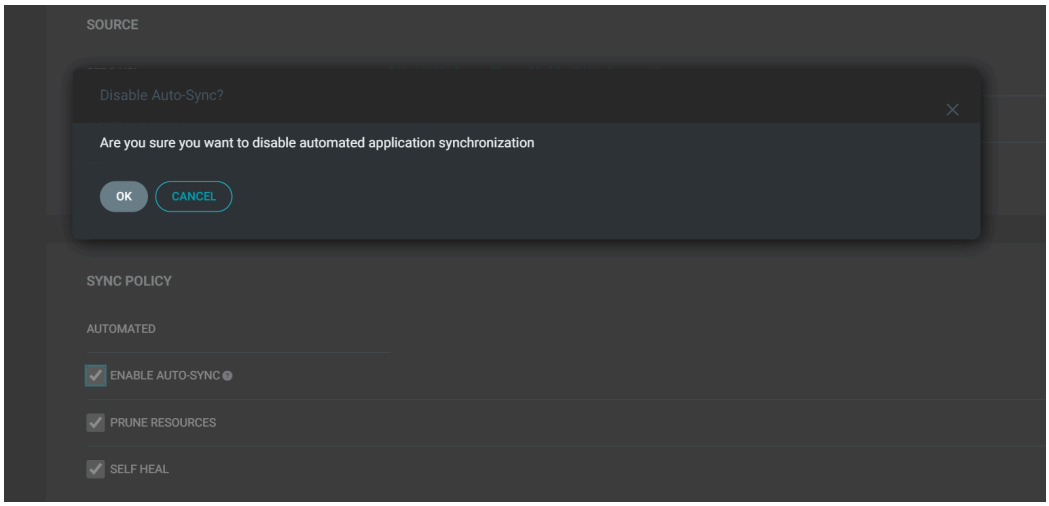


Fig. 26. Confirmation de la désactivation de la synchronisation automatique

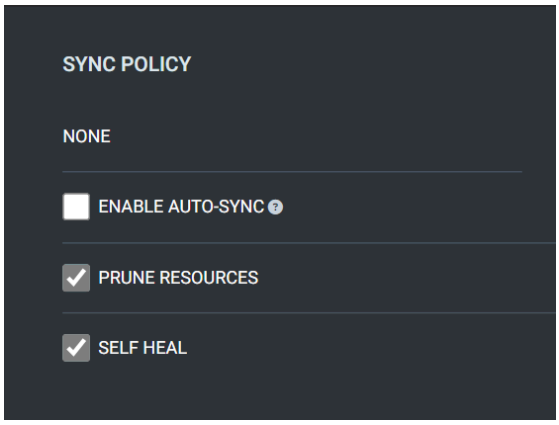


Fig. 27. Sync Policy passée à None : Prune et Self Heal restent cochés mais n'ont plus d'effet sans auto-sync

Historique des déploiements

L'onglet History and Rollback liste chaque synchronisation effectuée : révision Git, auteur, message de commit, date, durée et origine (automated sync policy ou utilisateur). L'Application conserve les 10 dernières entrées (`revisionHistoryLimit: 10`).

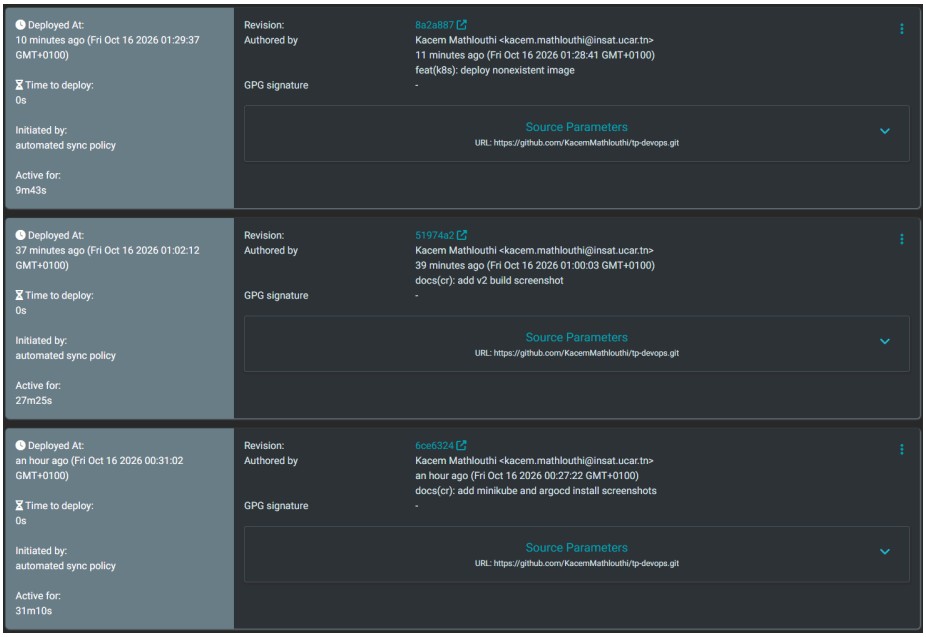


Fig. 28. Historique avant rollback : `8a2a887` (image inexistante, active), `51974a2` (version v2) et `6ce6324` (version v1)

> **Attention**
>
> La révision v2 apparaît sous le message `docs(cr): add v2 build screenshot` et non `feat(k8s): deploy v2` : les deux commits ont été poussés ensemble, et ArgoCD enregistre la tête de la branche au moment de la synchronisation. Le contenu déployé est bien celui de l'overlay avec `newTag: v2`.

Rollback

Depuis le menu de la révision `51974a2`, on choisit Rollback :

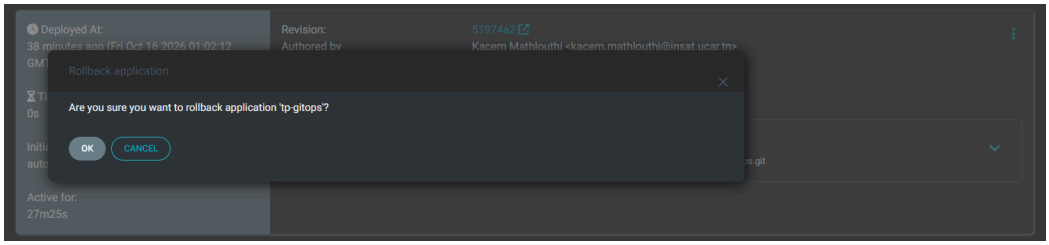


Fig. 29. Confirmation du rollback de l'application `tp-gitops` vers la révision `51974a2`

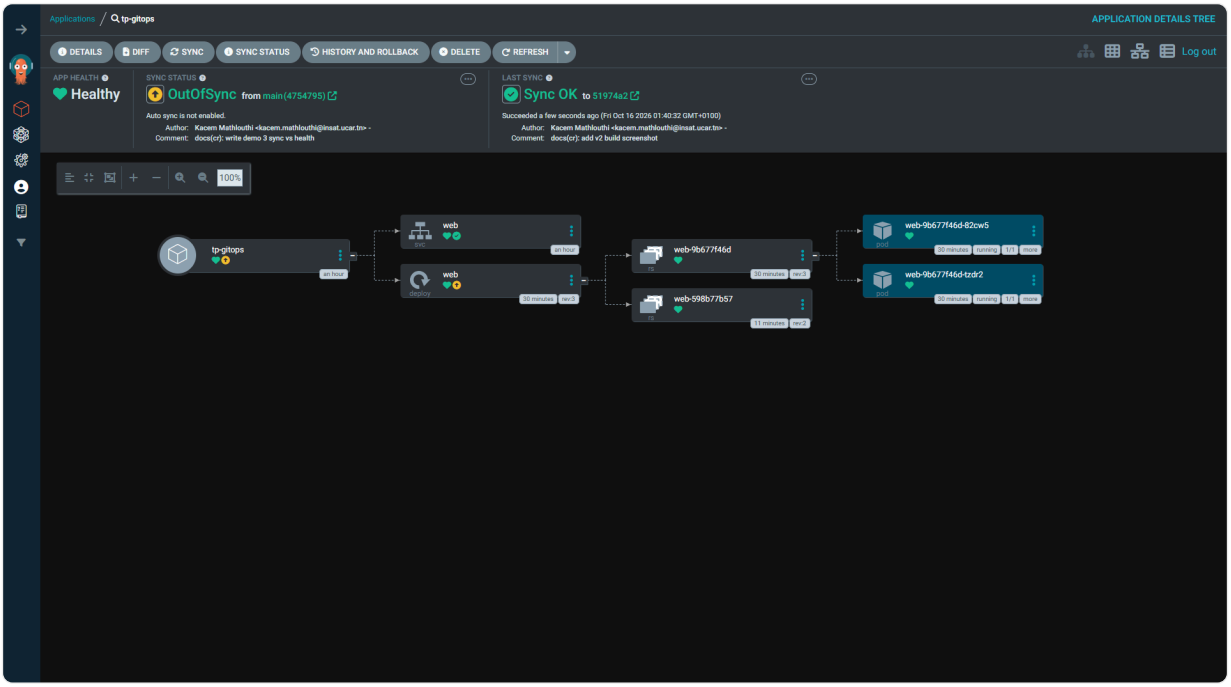


Fig. 30. Quelques secondes après le rollback : Healthy (Last Sync vers `51974a2`) mais OutOfSync par rapport à `main`, qui référence toujours l'image cassée

Le rollback est quasi instantané et ne crée même aucun pod : Kubernetes retrouve l'ancien ReplicaSet v2 (`web-9b677f46d`), dont les pods n'avaient jamais été arrêtés, et le promeut en révision `rev:3` ; le ReplicaSet défaillant est ramené à zéro.

L'état OutOfSync est le résultat attendu : le cluster ne correspond plus à Git. Le rollback depuis l'interface est une mesure d'urgence qui court-circuite temporairement la source de vérité.

Retour à Git comme source de vérité

Pour clore l'incident, l'erreur est annulée dans Git par un `git revert`, qui conserve la trace de l'erreur et de sa correction dans l'historique :

```
git revert 8a2a887       # newTag: doesnotexist → v2
git push
```

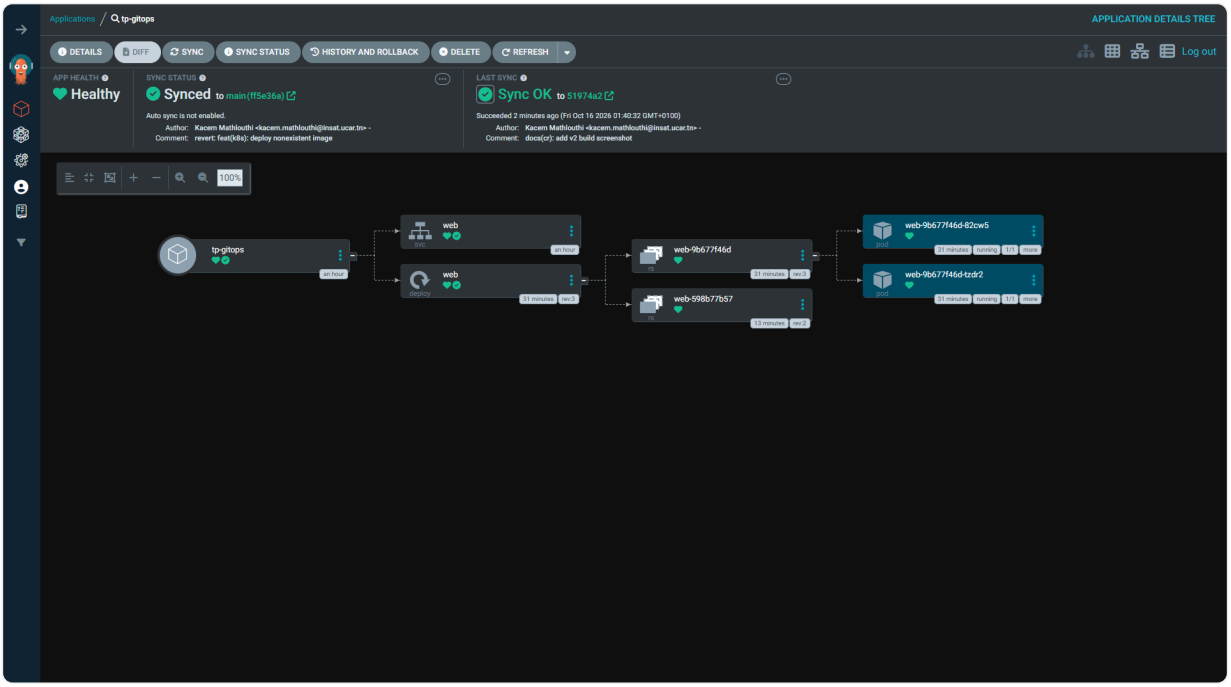


Fig. 31. Après le revert : Synced sur `ff5e36a` et Healthy, alors que l'auto-sync est encore désactivé. Git décrit à nouveau exactement ce qui tourne, aucune synchronisation n'est nécessaire

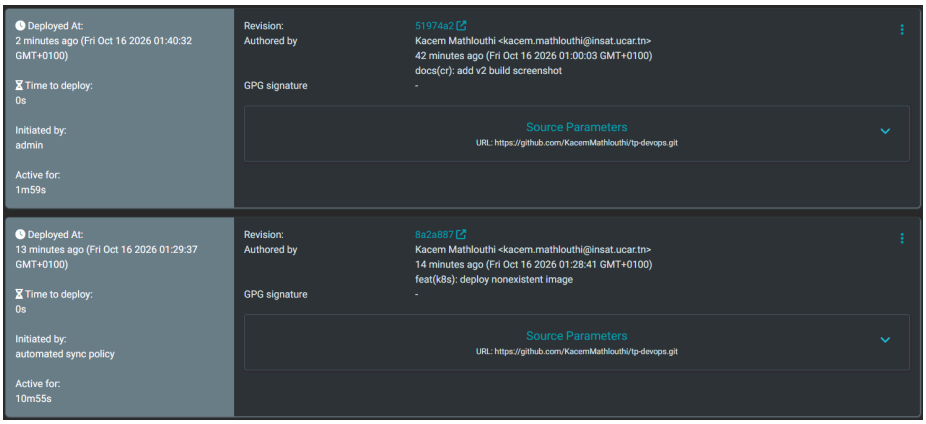


Fig. 32. Le rollback apparaît dans l'historique comme un nouveau déploiement de `51974a2`, initié par `admin` et non par la politique automatique

L'auto-sync est enfin réactivé : l'application retrouve son mode de fonctionnement GitOps nominal.

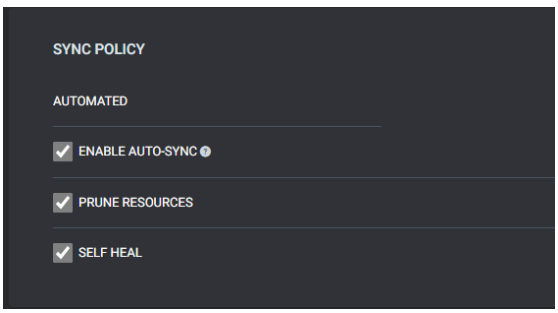


Fig. 33. Sync Policy de nouveau Automated, avec Prune et Self Heal

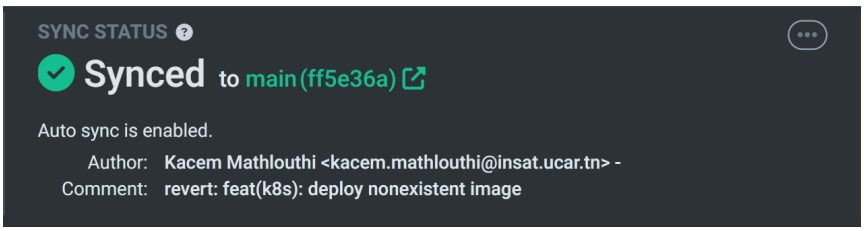


Fig. 34. État final : Synced sur le commit de revert `ff5e36a`, Auto sync is enabled

	Rollback ArgoCD (UI)	`git revert` + push
Délai	immédiat	détection du commit (polling ou Refresh)
Traçabilité	historique ArgoCD uniquement	historique Git, relisible et revu
Auto-sync	doit être désactivé	reste actif
Usage	mesure d'urgence	correction pérenne

- **Résultat**

L'historique ArgoCD a permis d'identifier la dernière version saine et d'y revenir en un clic : l'application est redevenue Healthy en quelques secondes. Le `git revert` a ensuite réaligné Git sur le cluster, puis l'auto-sync a été réactivé. L'état final est Synced et Healthy, avec Git de nouveau seule source de vérité.

Conclusion

Ce TP a mis en place une chaîne de déploiement continu entièrement pilotée par Git. Le code applicatif, la construction de l'image, les manifests Kubernetes et la définition de l'Application ArgoCD sont versionnés dans un même dépôt ; seule l'Application a été appliquée à la main, une fois.

Single Source of Truth	l'état désiré complet (Deployment, Service, namespace, version, réplicas) est décrit dans l'overlay Kustomize ; aucune ressource applicative n'a été créée avec `kubectl apply`.
Pull-based CD	la CI ne fait que publier des images ; c'est ArgoCD, dans le cluster, qui tire les changements de Git. Démo 1 : un commit d'une ligne a déployé la v2 par rolling update, sans interruption.
Drift & Self-Healing	démo 2 : un Deployment supprimé a été recréé en quelques secondes, un scale manuel annulé aussitôt.
Sync vs Health	démo 3 : une image inexistante donne une application Synced mais Degraded ; la synchronisation garantit la conformité à Git, pas le bon fonctionnement.
Rollback & Historique	démo 4 : retour en un clic à la dernière révision saine, puis `git revert` pour que Git redevienne la référence.

Ces démonstrations font aussi ressortir les limites et les points d'attention de l'approche :

- la détection repose sur le polling de Git (3 minutes par défaut) ; en production, un webhook GitHub vers ArgoCD rendrait la détection quasi immédiate ;
- le rollback depuis l'interface entre en tension avec le principe GitOps : il impose de suspendre l'auto-sync et doit toujours être suivi d'une correction dans Git ;
- ArgoCD ne valide pas ce qu'il déploie : une erreur dans Git est appliquée fidèlement. Des garde-fous en amont (revue de code, vérification de l'existence de l'image en CI) ou des stratégies progressives (Argo Rollouts, canary) complètent utilement le dispositif ;
- la structure base/overlay rend l'ajout d'un environnement (par exemple `overlays/prod` et une seconde Application) peu coûteux, la promotion d'une version se résumant alors à un changement de tag dans un autre overlay.

Dépôt

- GitHub : github.com/KacemMathlouthi/tp-devops